

01 - Introduction to Automata Theory

① Automata theory - study of abstract computing devices, or machines.

Automaton = an abstract computing device.

② What can computer do at all - this study is called decidability.
The problems that can be solved by computer are called decidable.

③ What can a computer do efficiently - this study is called intractability.

→ An alphabet is a ^{finite, non-empty} set of symbols: $\{0, 1\}$

→ Sentences are strings of symbols: $0, 1, 00, 01, 10, \dots$

→ A language is a set of sentences: $\{000, 0100, 1001, \dots\}$

→ A grammar can be regarded as a device that enumerates the sentences of a language.

Alphabet → An alphabet is a finite, non-empty set of symbols.

We use Σ ; Binary: $\Sigma = \{0, 1\}$

Alphanumeric: $\Sigma = \{a-z, A-Z, 0-9\}$

String → A string or word is a finite sequence of symbols.

Empty string is ϵ .

Length of string w , denoted by $|w|$

Powers of an alphabet: $\Sigma^0 = \{\epsilon\}$

if $\Sigma = \{0, 1\}$ then $\Sigma^1 = \{0, 1\}$ $\Sigma^2 = \{00, 01, 10, 11\}$

Σ^k = the set of all strings of length k .

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$ (set of all strings over an alphabet)

$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$ (set of nonempty strings)

The membership problem:

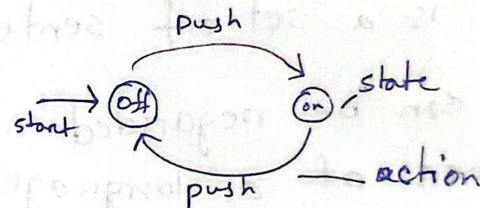
Given a string $w \in \Sigma^*$ and a language L over Σ , decide whether or not $w \in L$

Q: Is $w \in$ the language of strings with equal number of 0s and 1s

Applications of finite Automata:

1. Software for designing & checking the behaviour of digital circuits.
2. Lexical analyzer of a typical compiler.
3. Software for scanning large body of text.
4. Software for verifying systems of all types that have a finite number of states.

Example: on/off switch



Structural Expressions:

■ Grammars: Grammars are useful models when designing software. $E \rightarrow E + E$

■ Regular expressions:

↳ Denote the structure of data, especially text string. E.g.,

"Palo Alto CA"

$\rightarrow [A-Z][a-z]^*([A-Z][a-z]^*)^*[A-Z][A-Z]$

Deductive Proofs

A deductive proof consists of a sequence of statements whose truth leads us from some initial statement, called the hypothesis or the given statement, to a conclusion statement.

Ex: If H then C . We say that C is deduced from H .

Theorem 1.3: If $n \geq 4$, then $2^n \geq n^2$.

$$\text{If } n=4, \quad 2^4 = 16 = 4^2,$$

let's say, n increases by 1

$$\text{So, left side, } 2^{n+1} = 2^n \cdot 2$$

$$\text{Right side, } (n+1)^2 = n^2 + 2n + 1 = n^2 \left(\frac{n+1}{n} \right)^2$$

$$\text{for } 4, \quad \left(\frac{n+1}{n} \right)^2 = \left(\frac{4+1}{4} \right)^2 = 1.25$$

$$\text{for } 5, \quad \left(\frac{n+1}{n} \right)^2 = 1.2$$

$$\text{for } 6, \quad \left(\frac{n+1}{n} \right)^2 = 1.17$$

$$\text{Now, } \left(\frac{n+1}{n} \right)^2 = (1.25)^2 = 1.5625$$

Since $1.5625 < 2$, each time n increases above 4 +
left side 2^n grows more than the right side n^2 .

$$\text{So, } 2^n \geq n^2 \text{ for } n \geq 4 \quad \text{pm}$$

Theorem 1.4 :- If n is the sum of the square of four positive integers, then $2^n \geq n^2$.

	Statement	Justification
1.	$n = a^2 + b^2 + c^2 + d^2$	Given
2.	$a \geq 1; b \geq 1; c \geq 1; d \geq 1$	Given
3.	$a^2 \geq 1; b^2 \geq 1; c^2 \geq 1; d^2 \geq 1$	(2) & properties of arithmetic.
4.	$n \geq 4$	(2), (3), & properties of arithmetic.
5.	$2^n \geq n^2$	(4) & theorem 1.3

Theorem 1.5 :- Let S be a finite subset of some infinite set U . Let T be the complement of S with respect to U . Then T is infinite.

Proof: We know that, $S \cup T = U$ and S and T are disjoint.

$$\text{So, } \|S\| + \|T\| = \|U\|$$

Since S is finite, $\|S\| = n$ for some integer n , and since U is infinite, there is no integer p such that $\|U\| = p$.

So, assume that T is finite; that is, $\|T\| = m$ for some integer m . Then

$$\|U\| = \|S\| + \|T\| = n + m,$$

which contradicts the given statements that there is no integer p equal to $\|U\|$.

So, T is infinite.

Proof by Contradiction: It is more convenient to prove the statement "if H then C " by proving "if H and not C then". A proof of this type is called proof by contradiction.

Theorem 1.10

$$RU(S \cap T) = (RUS) \cap (RUT)$$

Proof: The two expressions involved are $E = RU(S \cap T)$

We shall prove two parts of the theorem in turn. In the "if" part, we assume element x is in E and show it is in F .

Then "only-if" part, we assume element x is in F and show it is in E .

Statement	Justification
1. x is in $RU(S \cap T)$	Given
2. x is in R or $S \cap T$	(1) & definition of Union
3. x is in R or x is in both S and T	(2) & definition of intersection
4. x is in RUS	(3) & definition of Union
5. x is in RUT	(3) & definition of Union
6. x is in $(RUS) \cap (RUT)$	(4), (5) & definition of Intersection

Step in the "if"

Statement	Justification
1. x is in $(RUS) \cap (RUT)$	Given
2. x is in RUS	(1) & definition of intersection
3. x is in RUT	(1) & definition of intersection
4. x is in R or x is in both S and T	(2), (3) & reasoning about unions
5. x is in R or x is in $S \cap T$	4 & definition of intersection
6. x is in $RU(S \cap T)$	(5) & definition of Union

Steps in the "only-if"

Finite Automata: Finite automata involve states and transitions among states in response to inputs.

Regular Expressions: These are a structural notation for describing the same patterns that can be represented by finite automata.

+ Inductive proofs: A statement that has an integer parameter n can often be proved by induction on n .

Example: (7.27) if $n \geq 4$

BASIS: If $n=4$, then 2^n and n^2 are both 16.

Thus, $2^4 \geq 4^2$ holds.

Induction: Suppose for some $n \geq 4$ that $2^n \geq n^2$.

We want to prove the $n \geq 4$ that $2^{n+1} \geq (n+1)^2$

We can write $2^{n+1} = 2 \times 2^n$

Since, $2^n \geq n^2$

$$2^{n+1} = 2 \cdot 2^n \geq 2n^2$$

But we need something different; we need to show that $2^{n+1} \geq (n+1)^2$

Now, we would like to prove,

$$2n^2 \geq (n+1)^2 \quad \text{--- (I)}$$

$$2n^2 \geq n^2 + 2n + 1$$

$$n \geq 2 + \frac{1}{n} \quad \text{--- (II)}$$

Since $n \geq 4$, ~~we know $\frac{1}{n} \leq \frac{1}{4}$~~ Thus the left side of (I) is at least 4, and the right side is at most 2.25.

We have thus proved the truth of (II) $\leq$

Define Finite Automata:

A finite automaton is an abstract computing machine that has a finite number of states and changes from one state to another in response to input symbols.

A finite automaton generally consists of:

1. States;
2. Input symbol;
3. Transitions;
4. Start state;
5. Accepting state.

Why study Finite Automata?

Finite automata are important in CS because they are useful models for many hardware and software systems. Their major applications include:

1. Digital circuits: Designing and checking the behaviour of digital circuits.
2. Compiler design: Implementing lexical analyzers that recognize identifiers, keywords and punctuation.
3. Text processing: Searching large collections of text for words, phrases and other patterns.
4. System verification: Verifying systems with a finite number of states, such as communication protocols.

2. Define Turing Machine -

A Turing machine is an abstract computational model introduced by Alan Turing to study what can and cannot be computed by a machine.

A Turing machine consists of;

- ① An infinite tapes
2. A read/write head.
3. Finite control
4. Transition rules.

Central concept of Automata theory:

The three central concepts of automata theory are

1. Alphabet (Σ): An alphabet is a finite, non-empty set of symbols.

Ex: $\Sigma = \{0, 1\}$; here, 0 & 1 are the symbols of binary alphabet.

2. String (w): A string is a finite sequence of symbols selected from an alphabet. The length of a string w is denoted by $|w|$.

Ex: $w = 1011$ $|w| = 4$

3. Language (L): A language is a set of strings formed from a particular alphabet. If Σ is an alphabet, then a language L satisfies

$$L \subseteq \Sigma^*$$

Ex: $\Sigma = \{0, 1\}$; $L = \{0, 01, 101, 111\}$

What is automata theory? Define decidability & intractability?

1. Automata theory is the branch of computer science that studies abstract computing devices, or machines and their computational capabilities and limitations.

Ex: Finite automata, Turing machine, pushdown automata.

2. Decidability is the study of whether a given problem can be solved by a computer in a finite amount of time.

Ex: Checking whether a given integer is even or odd.

3. Intractability is the study of problems that can be solved by a computer but require so much time or computational resources.

↳ Studies whether a problem can be solved efficiently.

Ex: The traveling salesman Problem.